

CFGS ASIR Gestión de Bases de Datos Curso 25 – 26	UT 4. SQL. MODIFICACIÓN EN BASES DE DATOS
Actividad de Aula	Boletín 5 - INSERT a partir de consultas

A partir del Dataset `dataset_ut4_dml.sql`, sobre alumnos, módulos, matrículas y calificaciones, realiza los siguientes ejercicios:

Instrucciones generales

En todos los ejercicios debes utilizar la sentencia `INSERT ... SELECT`.

Antes de ejecutar la inserción, **analiza si la operación es estructuralmente válida** teniendo en cuenta:

- número y orden de columnas
- tipos de datos
- claves primarias
- claves foráneas
- restricciones `NOT NULL`

Ejercicio 1

Se quiere insertar en la tabla `alumnos_beca` el identificador de todos los alumnos junto con el texto 'General' como tipo de beca.

Escribe la sentencia `INSERT ... SELECT` necesaria y **explica por qué la inserción es válida la primera vez**, pero **fallaría si se ejecuta dos veces seguidas**.

Ejercicio 2

Se intenta insertar en `alumnos_beca` únicamente el campo `id_alumno` obtenido desde la tabla `alumnos`.

Analiza por qué esta inserción **no es válida** desde el punto de vista estructural y **qué requisito del `INSERT ... SELECT` no se cumple**.

Ejercicio 3

Se desea insertar en `alumnos_beca` los campos `id_alumno` y `tipo_beca`, obteniendo ambos valores desde la tabla `alumnos`.

Explica por qué una consulta que seleccione `nombre` y `curso` **provoca un error**, indicando **qué problema de tipos de datos se produce**.

Ejercicio 4

Se plantea una inserción en `alumnos_beca` usando un `SELECT` que devuelve correctamente dos columnas, pero en orden inverso al esperado.

Describe **qué efecto tiene el orden de las columnas** en un `INSERT . . . SELECT` y por qué esta inserción **no sería correcta aunque el número de columnas coincida**.

Ejercicio 5

Se intenta insertar en `alumnos_beca` un valor fijo `NULL` en la columna `tipo_beca`.

Analiza por qué la base de datos **rechaza esta inserción** e indica **qué restricción se está violando**.

Ejercicio 6

Se ejecuta un `INSERT . . . SELECT` que devuelve correctamente filas, pero **no se inserta ninguna** en la tabla destino.

Explica por qué esta situación **no genera error**, pero tampoco inserta datos, y por qué **no contradice el funcionamiento de MariaDB**.

Ejercicio 7

Se quiere insertar en `alumnos_beca` un alumno cuyo `id_alumno` no existe en la tabla `alumnos`.

Explica por qué esta inserción **no está permitida**, indicando **qué papel juega la clave foránea**.

Ejercicio 8

Se plantea un `INSERT . . . SELECT` que devuelve más filas de las esperadas debido a una condición `WHERE` mal planteada.

Explica por qué el error **no está en el INSERT**, sino en la **consulta SELECT**, y cómo afecta eso al resultado final.

Ejercicio 9

Se intenta insertar registros duplicados en una tabla que tiene definida una clave primaria compuesta.

Explica por qué MariaDB **bloquea la inserción** y qué mecanismo de protección de la integridad entra en juego.

Ejercicio 10

Se ejecuta un `INSERT . . . SELECT` correctamente construido, pero uno de los valores devueltos por la consulta es `NULL` y la columna destino no lo admite.

Analiza por qué **toda la inserción falla**, aunque solo una fila incumpla la restricción.

Ejercicio 11

Se plantea una inserción usando una subconsulta en una condición **WHERE**, pero la subconsulta devuelve **más de una fila**.

Explica por qué esta situación provoca un error y **qué debería garantizar la subconsulta para ser válida**.

Ejercicio 12

Se intenta insertar datos en una tabla destino que tiene más columnas **NOT NULL** de las que se indican en el **INSERT**.

Analiza por qué esta inserción **no es válida**, aunque las columnas omitidas tengan valores definidos en otras filas.

Ejercicio 13

Se ejecuta un **INSERT . . . SELECT** en el que los tipos de datos son compatibles, pero **semánticamente incoherentes** (por ejemplo, textos en campos numéricos).

Explica por qué **el SGBD permite o rechaza** esta inserción y qué papel juega la definición del tipo de dato.

Ejercicio 14

Se realiza una inserción en una tabla con clave primaria simple, usando valores obtenidos de otra tabla.

Explica qué ocurre si el **SELECT** devuelve un valor que **ya existe como clave primaria** en la tabla destino.

Ejercicio 15

Se ejecuta un **INSERT . . . SELECT** que inserta correctamente varias filas, pero se produce un error en una de ellas.

Explica por qué MariaDB **no inserta ninguna fila** y cómo se comporta la operación desde el punto de vista transaccional.

Ejercicio 16

Se plantea una inserción en la que el número de columnas coincide, pero una de ellas **no respeta la longitud máxima del campo**.

Analiza por qué la inserción falla y **qué tipo de restricción se está incumpliendo**.

Ejercicio 17

Se utiliza un **INSERT . . . SELECT** sin especificar las columnas de destino.

Explica en qué casos esto es válido y **qué riesgo supone** si la estructura de la tabla cambia.

Ejercicio 18

Se intenta insertar datos en una tabla con una clave foránea, pero la tabla referenciada está vacía.

Explica por qué **ninguna fila puede insertarse** y qué relación existe entre ambas tablas.

Ejercicio 19

Se ejecuta un `INSERT . . . SELECT` correctamente estructurado, pero el resultado del `SELECT` depende de una condición `WHERE` que no se cumple para ningún registro.

Explica por qué **no se inserta nada**, pero **tampoco se produce error**.

Ejercicio 20

Tras analizar varios errores de inserción, se pide al alumnado que explique, con sus palabras, **por qué `INSERT . . . SELECT` es una operación estructural y no solo sintáctica**, apoyándose en ejemplos de los ejercicios anteriores.